

TP MQTT – Objets Connectés

Prérequis : try... except

Les exceptions sont des erreurs que Python peut rencontrer lors de l'exécution de votre programme.

```
1 >>> # Exemple classique : test d'une division par zéro
2 >>> variable = 1/0
3 Traceback (most recent call last):
4   File "<stdin>", line 1, in <module>
5   ZeroDivisionError: int division or modulo by zero
```

try ...except : Permet de mettre les instructions que l'on veut tester dans un premier bloc et les instructions à exécuter en cas d'erreur dans un autre bloc.

```
1 try:
2     # Bloc à essayer
3 except:
4     # Bloc qui sera exécuté en cas d'erreur
```

Exercice 1 : Broker local depuis Debian avec mosquitto sub/pub

Dans votre bash, utilisez la commande “man” pour lire le manuel en ligne de commande :

- mosquitto_pub
- mosquitto_sub

Depuis le serveur Debian (10.34.164.21 avec le port : 22), et en utilisant deux terminaux PUTTY (ou SSH sur MAC) utilisez les deux commandes ci-dessus pour transmettre un message “Hello world” via le topic MQTT : /UserXX/HW du broker local (127.0.0.1)

Exercice 2 : Broker local depuis Raspbian avec un script python

2-a) Faites la même chose que l'exercice 1 depuis le serveur Raspbian (10.34.164.22 avec le port : 22) mais en utilisant un script python à la place de la commande *mosquitto_pub* pour faire l'envoi.

Dans un deuxième terminal (toujours depuis votre serveur Raspbian), vous continuez à utiliser *mosquitto_sub* pour la récupération du message envoyé.

Exemple d'un script python qui publie le message "toto" sur le topic 'test' :

```
import paho.mqtt.client as mqtt

"""

La fonction publish() étant asynchrone, elle se termine avant
l'envoi réel du message. Pour cela, il faut donc utiliser
loop_forever() pour permettre au programme de tourner à l'infini
(pour gérer les événements MQTT). loop_forever() s'arrête quand on
exécutera la fonction loop_stop(). Dans cet exemple on crée une
fonction on_pub() qui sera appelée dès que l'événement de
publication aura lieu (c.à.d. client.on_publish = on_pub)

""" def

on_pub(client,userdata,result):

    print("message published")

    client.loop_stop()

    client.disconnect()

    client = mqtt.Client()

    client.on_publish = on_pub

    client.connect("127.0.0.1", 1883)

client.publish("test","toto",2) #2 correspond au QoS maximal

client.loop_forever()
```

2-b) Dans un terminal (toujours depuis votre serveur Raspbian), utilisez *mosquitto_pub* pour envoyer un message "Hello World".

Puis dans un deuxième temps, faites la même chose que l'exercice 1 depuis le serveur Raspbian (10.34.164.22 avec le port : 22) mais en utilisant un script python à la place de la commande *mosquitto_sub* pour faire la souscription et afficher les messages reçus.

Exemple d'un script python qui souscrit sue le topic test du broker local :

```
"""
La fonction subscribe () indique à quel topic on souhaite s'abonner,
ensuite via loop_forever() on attend et on traite les événements
MQTT.

Dans cet exemple, on crée une fonction on_mes() qui sera
appelée dès que l'événement de réception de message aura lieu
(c.à.d. client.on_message = on_mes). Dans la fonction après avoir
décodé le message et l'avoir affiché, on arrête la boucle de
traitement des événements et on se déconnecte client.loop_stop() et
client.disconnect() (comme dans le premier exemple)
"""

import paho.mqtt.client as mqtt

def on_mes(client, userdata,
mes):

    msg=msg.payload.decode("utf-8")

    print("Received ",msg," on topic ",mes.topic)

    client.loop_stop()

    client.disconnect()

client = mqtt.Client()

client.on_message = on_mes

client.connect("127.0.0.1", 1883)

client.subscribe("#",2)

client.loop_forever()
```

Exercice 3: *Broker gratuit avec 1 client Debian et 1 client Raspbian avec 2 scripts équivalent sub/pub*

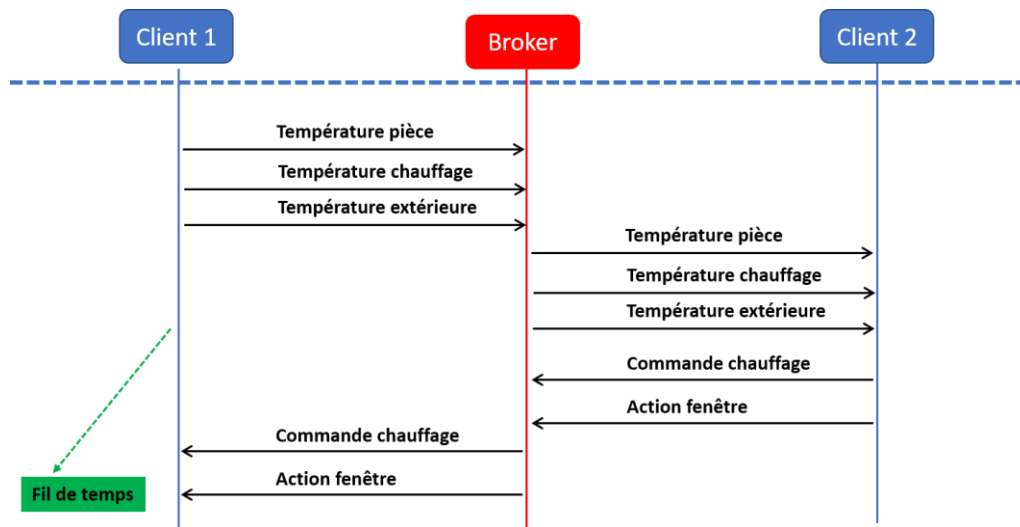
En utilisant SCP sur MAC et WINSCP sur Windows transférez le script de l'exercice 2-b vers le serveur Debian. L'opération se fait en deux fois.

Pour cet exercice, on n'utilise plus le broker local (127.0.0.1), il faut utiliser le broker MQTT public et gratuit (test.mosquitto.org).

Utilisez 2 scripts l'un sur le serveur Debian pour publier le message "Hello World" (sur le broker test.mosquitto.org), et l'autre sur le serveur Raspbian pour le récupérer du broker et l'afficher.

Exercice 4:

Le but de cet exercice est d'assurer une communication dans les 2 sens avec 2 scripts équivalents sub/pub en MQTT.



Le client 1 enverra 3 températures : pièce, chauffage, extérieur.

Le client 2 abonné au topics correspondants prendra la décision de couper le chauffage si la température dépasse les 25 degrés et/ou ouvrir la fenêtre s'il la température extérieur est plus grande que la température intérieure (s'il fait plus chaud à l'extérieur).

Utilisez 3 topics pour les températures et 2 topics pour les actions (chauffage/fenêtre) de la forme suivante :

- /Junia/Userxx/temp_piece
- /Junia/Userxx/temp_chauff
- /Junia/Userxx/temp_ext
- /Junia/Userxx/comm_chauff
- /Junia/Userxx/act_fen

Pour cet exercice, on n'utilise plus le broker local (127.0.0.1), il faut utiliser le broker MQTT mosquitto.junia.com.

Le client 1 sera sur le serveur Debian et le client 2 sur le serveur Raspbian.

Exercice 5: Utilisez JSON avec l'Ex 4 avec une donnée sérialisée dans 1 seul topic pour les 3 températures et 1 topic pour les commandes de chauffage et action fenêtre.

Utilisez les topics de la forme suivante :

- /Junia/Userxx/temp
- /Junia/Userxx/cmd_act